ใบงานที่ 4

เรื่อง การทดลองวงจรหน่วยประมวลผลคณิตศาสตร์และตรรกวิทยา

จัดทำโดย

นาย ชญานนท์ คำเป็ง

รหัส 68543206064-6

ห้อง ENGCE200_SEC1

เสนอ

อาจารย์สมนึก สุระธง

ใบงานนี้เป็นส่วนหนึ่งของวิชา

ENGCE200

การออกแบบระบบดิจิทัล

(Digital Systems Design)

หลักสูตร วศ.บ.วิศวกรรมคอมพิวเตอร์

สาขาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา ภาคพายัพ เชียงใหม่

ภาคเรียนที่ 1 ปีการศึกษา 2569

ใบงานที่ 4

ชื่อบทเรียน การทดลองวงจรหน่วยประมวลผลคณิตศาสตร์และตรรกวิทยา

จุดประสงค์การสอน ปฏิบัติการทดลองวงจรหน่วยประมวลผลคณิตศาสตร์และตรรกวิทยา

ออกแบบวงจรหน่วยประมวลผลคณิตศาสตร์และตรรกวิทยา

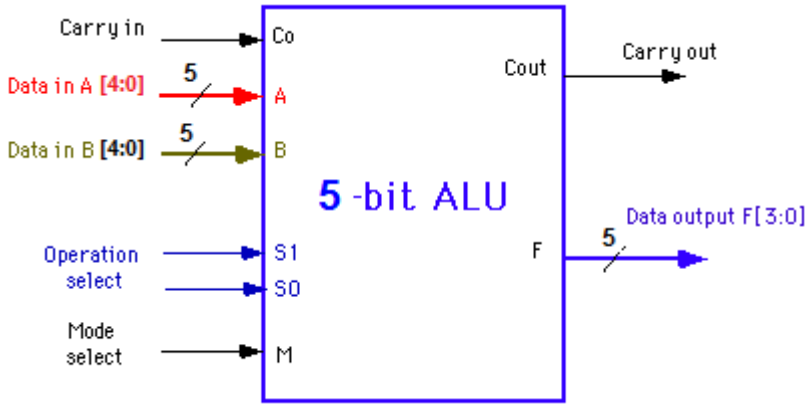
1. ทฤษฎี

วิธีการออกแบบวงจรบวก

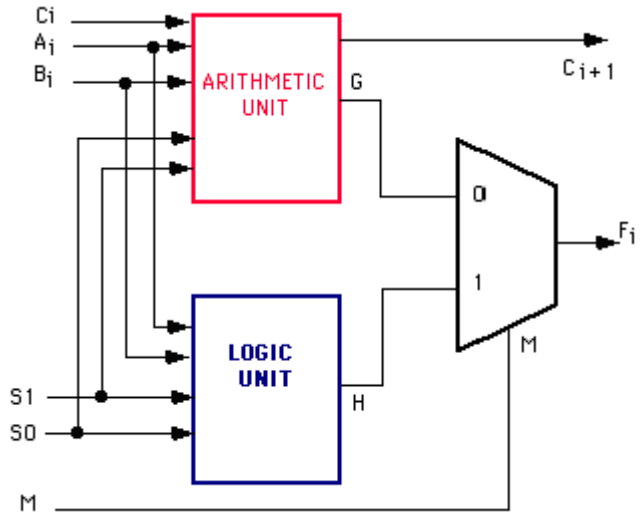
วิธีการออกแบบวงจรลบ

วิธีการออกแบบวงจรตรรกวิทยา

2. จงออกแบบวงจรวงจร 5 Bit ALU



รูปที่ 1 บล็อกไดอะแกรม 5 Bit ALU



รูปที่ 2 บล็อกไดอะแกรมภายใน 5 Bit ALU มีสองส่วน

ตารางที่ 1: Functions of ALU

M	S1	S0	C0	FUNCTION	OPERATION
0	0	0	0	A	Transfer A
0	0	0	1	A + 1	Increment A by 1
0	0	1	0	A + B	Add A and B
0	0	1	1	A + B + 1	Increment the sum of A and B by 1
0	1	0	0	A + B'	A plus one's complement of B
0	1	0	1	A - B	Subtract B from A (i.e. B' + A + 1)
0	1	1	0	A' + B	B plus one's complement of A
0	1	1	1	B - A	B minus A (or A' + B + 1)
M	S1	S0	C0	FUNCTION	OPERATION (bit wise)
1	0	0	X	AiBi	AND
1	0	1	X	Ai + Bi	OR
1	1	0	X	Ai$\oplus$ Bi	XOR
1	1	1	X	(Ai$\oplus$ Bi)’	XNOR

3. ผลการทดลอง

4. สรุปผลการทดลอง

5. คำถามหลังการทดลอง

1. การออกแบบวงจรบวกมีกี่วิธีอะไรบ่าง?
2. บอกหลักการทำวงจรบวกให้เป็นวงจรลบ
3. ถ้าต้องการออกแบบวงจรบวกและลบเลขขนาด 128 บิทจะต้องออกแบบอย่างไร

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity alu_4bit is
    Port (
        M, S1, S0 : in  STD_LOGIC;
        Cin_in : in  STD_LOGIC;
        A4, A3, A2, A1, A0 : in STD_LOGIC;
        B4, B3, B2, B1, B0 : in STD_LOGIC;
        Cin, GB, FB, GA, FA : out STD_LOGIC;
        Mini0, Mini1, Mini2, Mini3, Mini4 : out STD_LOGIC;
        C0, C1, C2, C3, C4 : out STD_LOGIC;
        F0, F1, F2, F3, F4 : out STD_LOGIC;
        Co : out STD_LOGIC
    );
end alu_4bit;
architecture Behavioral of alu_4bit is
    signal SEL : STD_LOGIC_VECTOR(1 downto 0);
    signal Cin_s : STD_LOGIC;
    signal GB_s  : STD_LOGIC;
    signal FB_s  : STD_LOGIC;
    signal GA_s  : STD_LOGIC;
    signal FA_s  : STD_LOGIC;
    signal X0, X1, X2, X3, X4 : STD_LOGIC;
    signal Y0, Y1, Y2, Y3, Y4 : STD_LOGIC;
    signal Carry0, Carry1, Carry2, Carry3, Carry4 : STD_LOGIC;
    signal M0, M1, M2, M3, M4 : STD_LOGIC;
begin

    -- Control Signals
    SEL <= S1 & S0;

    Cin_s <= (not M) and Cin_in;
```

```
GB_s  <= M or S1 or S0;
FB_s  <= (not M) and S1 and (not S0);
GA_s  <= '1';
FA_s  <= (not M) and S1 and S0;

Cin <= Cin_s;
GB  <= GB_s;
FB  <= FB_s;
GA  <= GA_s;
FA  <= FA_s;

-- Mini0

with SEL select
M0 <= (A0 and B0)  when "00",
      (A0 or B0)   when "01",
      (A0 xor B0)  when "10",
      (A0 xnor B0) when others;

-- Mini1

with SEL select
M1 <= (A1 and B1)  when "00",
      (A1 or B1)   when "01",
      (A1 xor B1)  when "10",
      (A1 xnor B1) when others;

-- Mini2

with SEL select
M2 <= (A2 and B2)  when "00",
      (A2 or B2)   when "01",
```

```
     (A2 xor B2)  when "10",
     (A2 xnor B2) when others;

-- Mini3

with SEL select
M3 <= (A3 and B3)  when "00",
     (A3 or B3)   when "01",
     (A3 xor B3)  when "10",
     (A3 xnor B3) when others;

-- Mini4

with SEL select
M4 <= (A4 and B4)  when "00",
     (A4 or B4)   when "01",
     (A4 xor B4)  when "10",
     (A4 xnor B4) when others;

Mini0 <= M0;
Mini1 <= M1;
Mini2 <= M2;
Mini3 <= M3;
Mini4 <= M4;

-- Arithmetic Generator

X0 <= FA_s xor (GA_s and A0);
Y0 <= FB_s xor (GB_s and B0);

X1 <= FA_s xor (GA_s and A1);
Y1 <= FB_s xor (GB_s and B1);
```

```
X2 <= FA_s xor (GA_s and A2);
Y2 <= FB_s xor (GB_s and B2);

X3 <= FA_s xor (GA_s and A3);
Y3 <= FB_s xor (GB_s and B3);

X4 <= FA_s xor (GA_s and A4);
Y4 <= FB_s xor (GB_s and B4);

-- Bit 0

F0 <= (Cin_s xor X0 xor Y0)
     when M='0'
     else M0;

Carry0 <= (Cin_s and (X0 or Y0))
        or
        (X0 and Y0);

C0 <= Carry0;

-- Bit 1

F1 <= (Carry0 xor X1 xor Y1)
     when M='0'
     else M1;

Carry1 <= (Carry0 and (X1 or Y1))
        or
        (X1 and Y1);
```

```
C1 <= Carry1;

-- Bit 2

F2 <= (Carry1 xor X2 xor Y2)
     when M='0'
     else M2;

Carry2 <= (Carry1 and (X2 or Y2))
        or
        (X2 and Y2);

C2 <= Carry2;

-- Bit 3

F3 <= (Carry2 xor X3 xor Y3)
     when M='0'
     else M3;

Carry3 <= (Carry2 and (X3 or Y3))
        or
        (X3 and Y3);

C3 <= Carry3;

-- Bit 4

F4 <= (Carry3 xor X4 xor Y4)
     when M='0'
     else M4;
```

```
    Carry4 <= (Carry3 and (X4 or Y4))
            or
            (X4 and Y4);

    C4 <= Carry4;

    -- Final Carry

    Co <= Carry4;

end Behavioral;
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity tb_alu_4bit is
end tb_alu_4bit;
architecture behavior of tb_alu_4bit is
    component alu_4bit
    Port (
        M, S1, S0 : in  STD_LOGIC;
        Cin_in : in  STD_LOGIC;
        A4, A3, A2, A1, A0 : in STD_LOGIC;
        B4, B3, B2, B1, B0 : in STD_LOGIC;
        Cin, GB, FB, GA, FA : out STD_LOGIC;
        Mini0, Mini1, Mini2, Mini3, Mini4 : out STD_LOGIC;
        C0, C1, C2, C3, C4 : out STD_LOGIC;
        F0, F1, F2, F3, F4 : out STD_LOGIC;
        Co : out STD_LOGIC
    );
    end component;
    signal M,S1,S0,Cin_in : STD_LOGIC;
    signal A4,A3,A2,A1,A0 : STD_LOGIC;
    signal B4,B3,B2,B1,B0 : STD_LOGIC;
    signal Cin,GB,FB,GA,FA : STD_LOGIC;
    signal Mini0,Mini1,Mini2,Mini3,Mini4 : STD_LOGIC;
    signal C0,C1,C2,C3,C4 : STD_LOGIC;
    signal F0,F1,F2,F3,F4 : STD_LOGIC;
    signal Co : STD_LOGIC;
begin
    DUT : alu_4bit
    port map(
        M => M,
        S1 => S1,
        S0 => S0,
        Cin_in => Cin_in,
        A4 => A4,
        A3 => A3,
```

```
    A2 => A2,
    A1 => A1,
    A0 => A0,
    B4 => B4,
    B3 => B3,
    B2 => B2,
    B1 => B1,
    B0 => B0,
    Cin => Cin,
    GB => GB,
    FB => FB,
    GA => GA,
    FA => FA,
    Mini0 => Mini0,
    Mini1 => Mini1,
    Mini2 => Mini2,
    Mini3 => Mini3,
    Mini4 => Mini4,
    C0 => C0,
    C1 => C1,
    C2 => C2,
    C3 => C3,
    C4 => C4,
    F0 => F0,
    F1 => F1,
    F2 => F2,
    F3 => F3,
    F4 => F4,
    Co => Co
);
process
begin
    -- A = 10101 1+4+16 = 21
    -- B = 01100 4+8 = 12
```

```
A4 <= '1';
A3 <= '0';
A2 <= '1';
A1 <= '0';
A0 <= '1';

B4 <= '0';
B3 <= '1';
B2 <= '1';
B1 <= '0';
B0 <= '0';

-- A
M <= '0'; S1 <= '0'; S0 <= '0'; Cin_in <= '0';
wait for 20 ns;

-- A+1
M <= '0'; S1 <= '0'; S0 <= '0'; Cin_in <= '1';
wait for 20 ns;

-- A+B
M <= '0'; S1 <= '0'; S0 <= '1'; Cin_in <= '0';
wait for 20 ns;

-- A+B+1
M <= '0'; S1 <= '0'; S0 <= '1'; Cin_in <= '1';
wait for 20 ns;

-- A+B'
M <= '0'; S1 <= '1'; S0 <= '0'; Cin_in <= '0';
wait for 20 ns;

-- A-B
M <= '0'; S1 <= '1'; S0 <= '0'; Cin_in <= '1';
wait for 20 ns;
```

```
        -- A'+B
        M <= '0'; S1 <= '1'; S0 <= '1'; Cin_in <= '0';
        wait for 20 ns;

        -- B-A
        M <= '0'; S1 <= '1'; S0 <= '1'; Cin_in <= '1';
        wait for 20 ns;

        -- AND
        M <= '1'; S1 <= '0'; S0 <= '0'; Cin_in <= '0';
        wait for 20 ns;

        -- OR
        M <= '1'; S1 <= '0'; S0 <= '1'; Cin_in <= '0';
        wait for 20 ns;

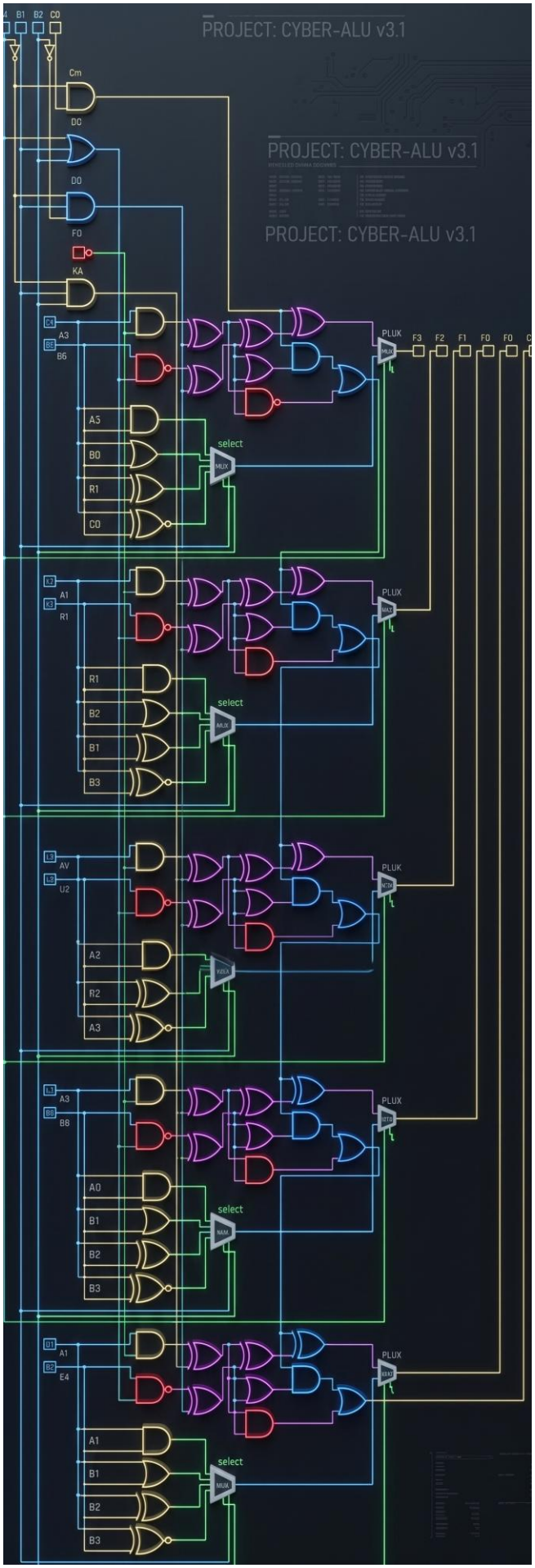
        -- XOR
        M <= '1'; S1 <= '1'; S0 <= '0'; Cin_in <= '0';
        wait for 20 ns;

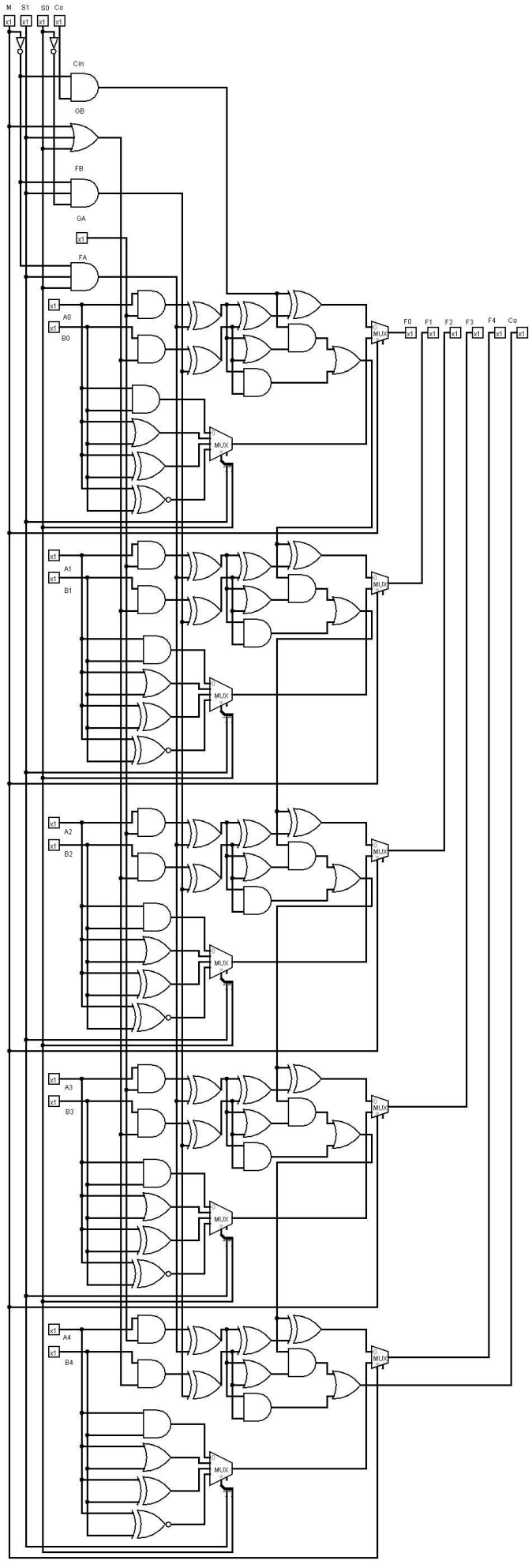
        -- XNOR
        M <= '1'; S1 <= '1'; S0 <= '1'; Cin_in <= '0';
        wait for 20 ns;

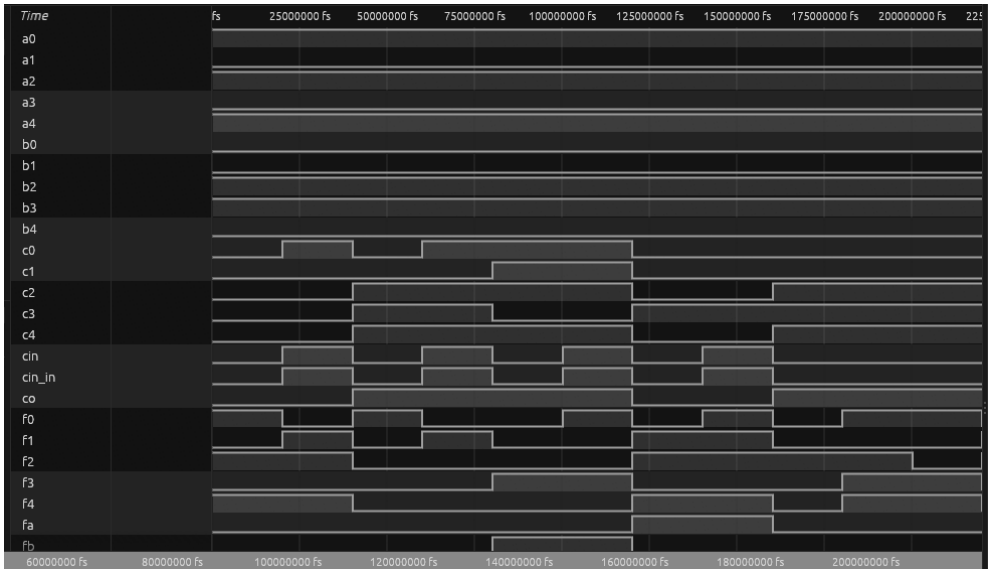
        wait;

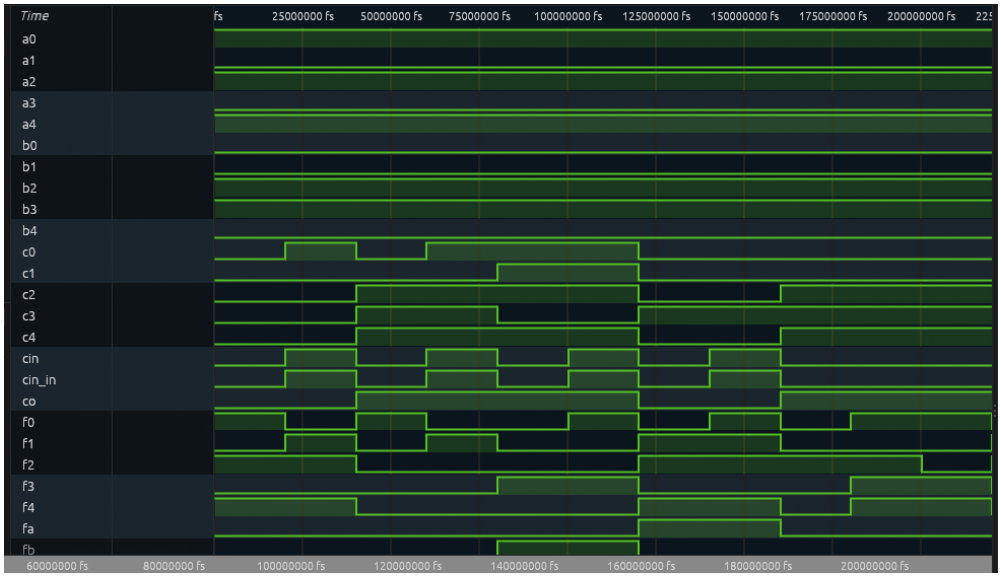
    end process;

end behavior;
```


PROJECT: CYBER-ALU v3.1
PROJECT: CYBER-ALU v3.1
PROJECT: CYBER-ALU v3.1
B1
B2
C0
Cm
D0
D0
F0
KA
A3
B6
A5
B0
R1
C0
select
PLUK
F3
F2
F1
F0
F0
A1
R1
R1
B2
B1
B3
select
PLUK
AV
U2
A2
R2
A3
PLUK
A3
BB
A0
B1
B2
B3
select
PLUK
A1
E4
A1
B1
B2
B3
select
PLUK


M
S1
S0
Co
Cin
GB
FB
GA
FA
A0
B0
A1
B1
A2
B2
A3
B3
A4
B4
MUX
F0
F1
F2
F3
F4
Co


Time
fs
25000000 fs
50000000 fs
75000000 fs
100000000 fs
125000000 fs
150000000 fs
175000000 fs
200000000 fs
a0
a1
a2
a3
a4
b0
b1
b2
b3
b4
c0
c1
c2
c3
c4
cin
cin_in
co
f0
f1
f2
f3
f4
fa
fb
60000000 fs
80000000 fs
100000000 fs
120000000 fs
140000000 fs
160000000 fs
180000000 fs
200000000 fs


Time
fs
25000000 fs
50000000 fs
75000000 fs
100000000 fs
125000000 fs
150000000 fs
175000000 fs
200000000 fs
a0
a1
a2
a3
a4
b0
b1
b2
b3
b4
c0
c1
c2
c3
c4
cin
cin_in
co
f0
f1
f2
f3
f4
fa
fb
60000000 fs
80000000 fs
100000000 fs
120000000 fs
140000000 fs
160000000 fs
180000000 fs
200000000 fs